

Università di Catania

Department of Economics and Business

Department of Electrical Electronic and Computer Engineering

Department of Mathematics and Computer Science

Master's Degree in Data Science

HDA: A Natural-Language Interface for Healthcare Data Analysis

MASTER DEGREE THESIS

Author
Thomas Alfio Drury

Supervisor
Professor Giovanni Morana

ACADEMIC YEAR 2025/2026
29 September 2026

Thomas Alfio Drury

HDA: A Natural-Language Interface for Healthcare Data Analysis

Master's Degree Thesis

UNIVERSITY OF CATANIA

September 2026

Contents

List of tables ..... *iii*

List of figures ..... *iv*

Introduction *1*

Chapter 1 - Background and State of the Art *3*

1.1 Hospital administrative data in Italy: SDO, ICD and DRG **3**
1.1.1 Why administrative data behaves the way it does 3
1.1.2 The Scheda di Dimissione Ospedaliera 3
1.1.3 From regulation to dataset: the 2024-2025 flow 5

1.2 Natural language interfaces to databases **6**
1.2.1 Definition and early systems 6
1.2.2 Difficulties that never went away 7
1.2.3 The neural formulation 8
1.2.4 Schema linking and logical synthesis 8
1.2.5 What the usability evidence says 9

1.3 Language models, tool use and agent-based systems **9**
1.3.1 From text generation to action 9
1.3.2 Protocols for tool access 10
1.3.3 Multi-agent Text-to-SQL 10
1.3.4 Agentic analytics in production 11
1.3.5 Reading the trend 11

1.4 Language models in healthcare analytics **12**
1.4.1 Conversational access to clinical databases 12
1.4.2 Adapting smaller models 12
1.4.3 Conversational analysis and visualisation 13
1.4.4 Clinical knowledge and its limits 14
1.4.5 Privacy as a first-class threat model 14

1.5 The research gap **15**
1.5.1 The accuracy ceiling, and the drop from benchmark to deployment 15
1.5.2 The evaluation problem 15
1.5.3 Hallucination and the difficulty of detecting it 16
1.5.4 Deployment assumptions 17
1.5.5 Statement of the gap 17

Chapter 2 - System Architecture *19*

2.1 System overview and the Text-to-API paradigm **19**
2.1.1 The paradigm stated 19
2.1.2 The request path 20
2.1.3 Why this decomposition 22

2.2 Data model and semantic layer **23**
2.2.1 Ingestion: absorbing the artefacts of the flow 23
2.2.2 The semantic model 24
2.2.3 One declaration, four responsibilities 26
2.2.4 The action catalogue 26

2.3 Intent routing and the controlled plan language **28**
2.3.1 The Known/Unknown partition 28
2.3.2 The Known branch: schema linking into an intent 29

2.3.3 The Unknown branch: a controlled plan language 30
2.3.4 Topic packs: deterministic domain knowledge 30

2.4 Safety, Validation and Deterministic Execution 31
2.4.1 The trust boundary 31
2.4.2 Failing closed 32
2.4.3 Structural guarantees 32
2.4.4 The minimum group size 32
2.4.5 Determinism and the test suite 33
2.4.6 The human in the loop: confirmation as supervision 34

2.5 User interface and analytics layer 36
2.5.1 Deterministic chart selection 37
2.5.2 Proactive secondary analysis 37
2.5.3 Drill-down: OLAP interaction without the LLM 38
2.5.4 Follow-up suggestions 39
2.5.5 The interface as an audit surface 39

Chapter 3 - Evaluation and Limitations 40

3.1 Evaluation dataset 40
3.1.1 Dataset 40
3.1.2 Lack of External Validation 40
3.1.3 The question set 40
3.1.4 Evaluation criteria 42
3.1.5 Evaluation results 42
3.1.6 Failure analysis 44
3.1.7 Representative questions 46

3.2 Effort and performance 47
3.2.1 Analyst effort 47
3.2.2 Performance 47

3.3 Limitations 48

3.4 Threats to validity 49

Chapter 4 - Contributions and Future Work 51

4.1 Contributions 51
4.1.1 What was built, and what it delivers 51

4.2 Future Developments 52
4.2.1 The gap: clinical language and database codes 52
4.2.2 The proposal: a category layer between vocabulary and codes 53
4.2.3 Where the categories come from: what already exists 55
4.2.4 What only clinicians can supply 56
4.2.5 Other directions 57

Conclusion 59

Appendix A 61

Appendix B 64

References 65

List of tables

Table 1: SDO Structure 4
Table 2: Dataset Structure 6
Table 3: Pipeline 22
Table 4: Semantic descriptions 24
Table 5: Dimensions 25
Table 6: The action catalogue 27
Table 7: Trust Boundary 31
Table 8: Test coverage of the deterministic layer 34
Table 9: Chart selection rules, in evaluation order 37
Table 10: Family question set 41
Table 11: Results by family (individual queries in Appendix A) 43
Table 12: Failure classes 44
Table 13: Example of questions/inputs 46
Table 14: Measures 47
Table 15: Execution times 48

List of figures

Figure 1: HDA Structure 20
Figure 2: The Known/Unknown partition 29
Figure 3: Minimum group size 33
Figure 4: The inspection and confirmation panel. 35
Figure 5: Streamlit output example 36
Figure 6: Examples of Olap operations 38
Figure 7: Follow-up questions 39

Introduction

Each hospital in Italy produces a discharge document called a *Scheda di Dimissione Ospedaliera* (SDO) for every patient. This document is a structured summary of the hospital stay, it states who the patient was, when they arrived and left, the principal diagnosis and procedure coded according to the current ICD-9-CM (International Classification of Diseases – 9 Clinical Modification), the DRG (Diagnosis Related Group) assigned to the episode, and finally, in what way the hospital experience concluded. These records are stored in a database that will be used for future hospital planning and analysis.

The people who need answers from that database are doctors, ward managers and quality managers. Very few of them know how to directly access the data from the database. Furthermore, the data is also sensitive: health data is a special category under the General Data Protection Regulation (GDPR), and the Italian regulation governing the SDO flow keeps identifying information in a separate archive from clinical content. The result highlights a well-known limitation. Questions are routed to an analytics team, generally having no domain expertise which results in significant delays between the time a hospital worker submits a question and the time an answer is delivered.

A natural language interface is the obvious solution. The interesting part of the question is not simply whether such an interface is desirable, it is but more importantly how to build one that is safe with clinical data, and qualitatively how to make it more than a simple text box accessing a query engine.

The state-of-the-art approaches available do not adequately fulfil this aspect, and they tend to fail for one shared reason: the language model's output is executable code. Text-to-SQL turns a question into a SQL (Structured Query Language) statement and runs it. Code-interpreter designs let the model write and execute a program over the data. Both are powerful. Both give the user an artefact that only a programmer can check. A non-technical user cannot distinguish between a correct query from a subtly wrong one, and a wrong join or a wrong level of aggregation does not

raise an error - it returns a plausible number which in the area of healthcare is potentially very dangerous.

The accuracy figures make the problem concrete. On BIRD, the reference benchmark for real life databases, the strongest published systems reach roughly 72 % execution accuracy. Roughly one question in four is therefore answered incorrectly rather than declined. On a real SDO flow - Italian column names, ministerial codes, formatting artefacts inherited from a fixed-width transmission format - there is no reason to expect that ceiling to be higher, and the errors that remain are the silent (or undetectable) kind.

This thesis proposes and implements HDA, Healthcare Discharge Analytics: an Italian-language analytics engine over SDO data, driven by a locally hosted quantised model, built on a different premise. The model never emits executable code. It emits either a validated intent naming a pre-built, pre-tested analytical action together with its parameters, or a plan drawn from a relatively small, fixed grammar restricted to columns declared in a semantic model.

This is what the thesis calls the Text-to-API (Application Programming Interface) approach. The question is translated into a choice among audited capabilities rather than into code, and the correctness burden shrinks accordingly: from *Did the model write a correct query over the whole schema?* to *Did the model pick the right pre-tested action?* The second question is bounded and can be checked one case at a time.

A second constraint shapes the design. Because the data is sensitive, nothing may be sent to an external service, so the whole pipeline runs on a local quantised AI model of moderate size rather than a strong cloud model. No clinical record ever leaves the machine.

Around this core, the system is deliberately proactive. From the user questions, the program generates automatically the primary chart or a summary of the main information, adds secondary charts on other dimensions, writes a summary in plain Italian language of the query, and suggests related questions the user may be interested in. The answers a user has inspected and confirmed are stored and replayed or reused, so repeated questions become faster and model-free.